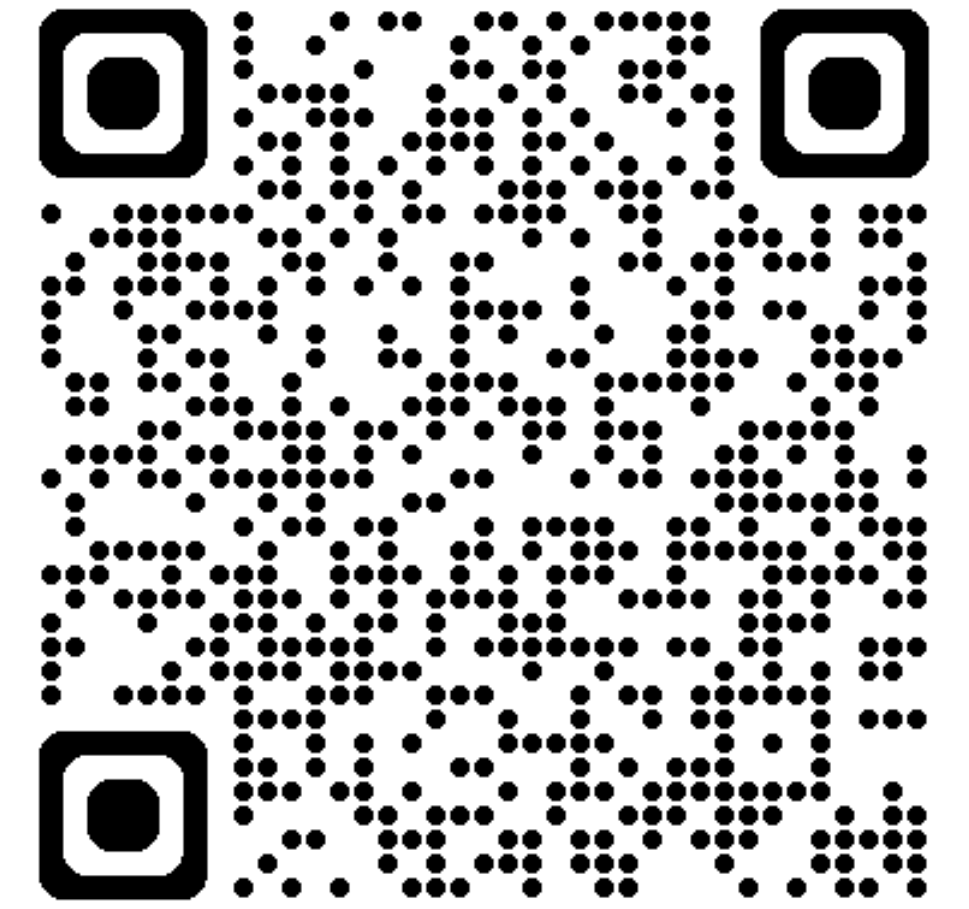


# Übungsstunde 3

## Heute: Volles Programm

- Rückblick & Repetitions Quiz
- Containers 2/2: Collections
- Aliasing: Erklärung „alles ist ein Pointer“
- Numpy
- Rekursion
- Ausblick Code Expert: Python II



# ...Letzte Woche

## Containers, Sequences

- List slicing
- Methoden auf listen
- Unterscheidung Anwendungsfälle für verschiedene Container
- **Kommentar zu CodeExpert:**
  - mehrheitlich gut gelöst worden, indiv. Feedback folgt die kommenden Tage
  - kommentiert gerne euren Code für euch. Hilfreich in der Lernphase schnell wieder die Prinzipien & Gedankengänge nachvollziehen zu können

# Repetitionsquiz I

## w1&2

1.

```
# 30 sekunden  
l=[3, 1, 4, 1, 5, 9]  
print(l.sort())  
print(l[::-1])  
print(type(l))  
print(dir(l))
```

2.

```
#30 sekunden  
x=[1, 2, 3,4]  
y=[4, 5, 6]  
print(list(zip(x,y)))
```

3.

```
y=[4, 5, 6]  
z=y  
z[0]=10  
print(z==y)
```

4.

```
#am besten mit notizen lösen, 1-2 minuten  
l=[3, 1,5]  
n = 2  
d={8000:'zurich'}  
for i in range(n + len(l)):  
    print(l[i % len(l)])  
    key=l[i % len(l)]  
    d[key]=i  
print(d)
```



# List Comprehension

erwünschte Listen mit nur einer Zeile erstellen

- kennt ihr bisher auf diese Weise:
- In Python geht dies noch eleganter mit der Comprehension

```
#mit for schleife eine list von 1 bis 10 erstellen
liste = []
for i in range(1, 11):
    liste.append(i)
print('using default for loop',liste)
```

```
#now with list comprehension
liste = [i for i in range(1, 11)]
print('using list comprehension', liste)
```

✓ 0.0s

using default for loop [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]  
using list comprehension [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

**beides** funktioniert; Versucht euch mit comprehensions vertraut zu machen  
**an der Prüfung:** verwendet das, womit ihr euch am wohlsten fühlt



# Syntax

```
new_list = [expression for item in iterable if condition]
```

- es wird **eine neue Liste erstellt**
- **item:** Stellt euch vor:  
genau so ist item auch verwendbar
- **expression:** der (Rückgabe)-Wert, welcher „in die Liste eingefügt wird“
  - bedeutet, wir können auch eine Funktion als Expression verwenden
  - ist anfangs etwas ungewöhnlich, da „item“ beim Lesen vor dem „**for**“ stehen kann.
- **condition:** eine Bedingung die erfüllt sein muss, damit die expression eingefügt wird in die neue liste

# expression und condition...

**Können eine Funktion von item sein, müssen es aber nicht!**

```
new_list = [expression for item in iterable if condition]
```

Funktionen von item

```
new_list=[item**2 for item in liste if item % 2 == 0]  
new_list=[item for item in liste if check_condition(item)]  
new_list=[func(item) for item in liste if check_condition(item)]
```

keine Funktionen von item

```
new_list=[10 for item in liste if True]  
new_list=[len(liste) for item in liste if True]  
new_list=[10 for item in liste if len(liste)>5]
```

**beachte:** natürlich gilt auch hier, der „Name“ für item kann beliebig gewählt werden

# Listen-Abstraktion: Quiz

Was ist die Ausgabe des folgenden Codes?

```
[x**2 for x in range(2,7)]
```

Wie kann man die folgende Liste mittels Listen-Abstraktion generieren?

```
[1, 2, 4, 8, 16, 32, 64, 128]
```



# Gefilterte Listen-Abstraktion: Quiz

Was ist die Ausgabe des folgenden Codes?

```
[x**3 for x in range(6) if x%2==1]
```

Wie kann man die folgenden Listen mittels gefilterter Listen-Abstraktion generieren?

```
[25, 16, 9, 4, 4, 9, 16, 25]
```

# Dictionaries

reminder: ein Dict order ein value einem key zuy:

simpel: postleitzahlen

```
postleitzahlen = {8000: 'zurich', 3000: 'berne', }
```



**ethzurich**  ...

ETH Zurich

1.893 Beiträge 139.000 Follower 346 Gefolgt

Where the future begins    
One of the world's leading universities  
for technology & natural sciences.

[ethz.ch/en](https://ethz.ch/en) und 4 weitere

```
instagram_profil = {  
    'username': 'ETH Zurich',  
    'followers': 139000,  
    'following': 346,  
    'posts': 1893,  
    'bio': 'where the future begins etc..'  
}
```

# klassische Operationen auf Dicts

Wert abfragen

```
ethz_biography = instagram_profil['bio']  
post_count = instagram_profil['posts']
```

Paar hinzufügen

```
ethz_ig_profile['website'] = 'https://ethz.ch'
```

Wert ändern

```
#update followers  
ethz_ig_profile['followers'] = 140000
```

key enthalten?

```
'followers' in ethz_ig_profile
```

key:value paar löschen

```
del ethz_ig_profile['following']
```



# zip() verwenden um ein dict zu erstellen

`dict(zip(keys, values))`

fertiges dictionary

keys



values

# vorteil gegenüber einer for loop

## nicht nur eleganter

- sondern: kommt klar mit zwei **nicht gleich lange Listen**
- **Szenario:** Daten zu einem Jahr dauern 6 Jahre, bis sie veröffentlicht werden
  - unsere Liste mit den Jahreszahlen ist länger als die verfügbaren Daten

```
#hdi over the last 5 years in switzerland, values in a list, years in another list

years = [2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024, 2025]
hdi_values = [0.96, 0.97, 0.98, 0.99]

#dictionary erstellen, in dem die jahre die keys sind und die hdi werte die values using zip
hdi_dict = dict(zip(years, hdi_values))
print(hdi_dict)
```

```
{2016: 0.96, 2017: 0.97, 2018: 0.98, 2019: 0.99}
```

wir „schneiden“ die überstehenden Jahre einfach ab  
dies bedeutet unser code ist unabhängig von der längendifferenz funktional :)

**beachte:** sollte unsere key-Liste doppelte einträge haben, wird **überschrieben**



# Anmerkung: über Zip iterieren

Nützlich in einer CX, Python II

```
#iterate over a zip object  
x=[1, 2, 3,4]  
y=[4, 5, 6]  
for a, b in zip(x, y):  
    print(a, b)
```

✓ 0.0s

1 4

2 5

3 6

# dict comprehension

gleiches Spiel wie bei Listen

**Syntax:** `{key_expression: value_expression for item in iterable if condition}`

erneut gilt:

- die **Expressions** können Funktionen von „item“ sein
- und auch hier können wir eine **Bedingung** stellen, damit ein paar hinzugefügt wird.

# Comprehensions mit zip() oder aus einem Dict

**beispiel:** Angenommen, wir haben eine Unsicherheit, unsere hdi-werte sind um .01 zu tief

und wir wollen ein **neues Dictionary** mit korrigierten Werten erstellen

aus einem anderen dict heraus

```
d = {f(k):g(v) for k, v in d0.items()}
```

mit zip() und zwei listen

```
d = {f(x):g(y) for x, y in zip(cx, cy)}
```

```
#dict comprehension nutzen um die hdi werte zu erhöhen um 0.01
hdi_dict = {year: hdi + 0.01 for year, hdi in hdi_dict.items()}
print('corrected hdi_dict with .items: ', hdi_dict,)
```

```
#dict comprehension mit zip() nutzen um die hdi werte zu erhöhen um 0.01
hdi_dict = {year: hdi + 0.01 for year, hdi in zip(years, hdi_values)}
print('corrected hdi_dict with zip: ', hdi_dict,)
```

✓ 0.0s

old hdi: {2016: 0.96, 2017: 0.97, 2018: 0.98, 2019: 0.99}

corrected hdi\_dict with .items: {2016: 0.97, 2017: 0.98, 2018: 0.99, 2019: 1.0}

corrected hdi\_dict with zip: {2016: 0.97, 2017: 0.98, 2018: 0.99, 2019: 1.0}



# Containers

weiter mit Aliasing, „Python under the hood“

# Alles ist ein Pointer in Python

## Merken:

- Variablen sind nur Namen, sie zeigen auf ein eigentliches Objekt/Wert
- **Aliasing** wenn wir einer Variable eine andere zuweisen, erstellen wir eigentlich nur einen weiteren Pointer auf das zugrundeliegende Objekt
- wir können die Speicher-adresse eines Objektes mit `id(obj)` abfragen
  - wir sehen, beide variablen zeigen auf dieselbe adresse

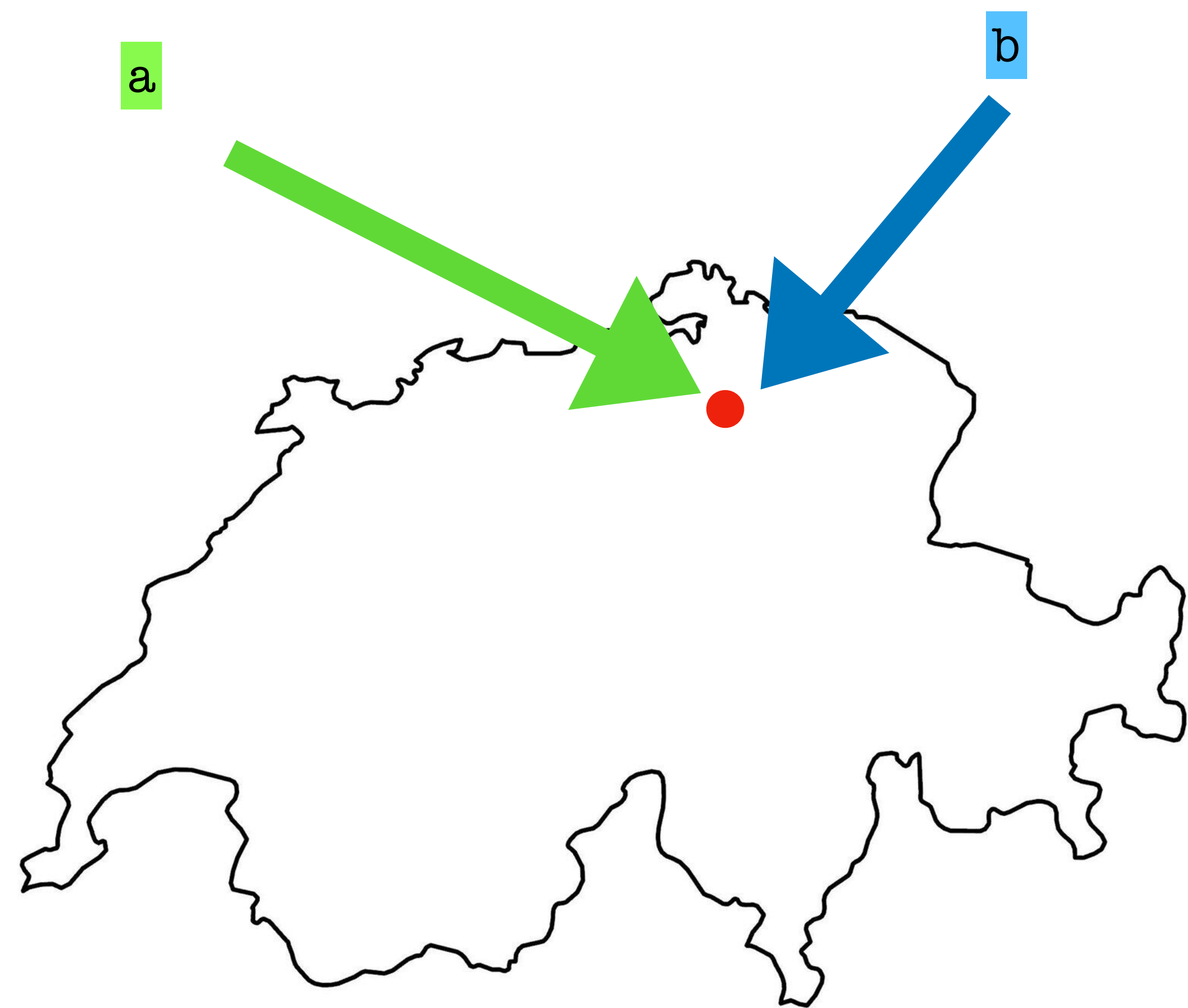
```
#is the is operator the same as the == operator?  
a = [47.3769, 8.5417] #coordinate of zurich  
b = a  
print(id(a)==id(b)) #alternativ: print(a is b)
```

✓ 0.0s

True

# Grafisch

für uns



im Computer

angenommen, er speichert unsere Objekte in einem solchen Raster.  
Dann nehmen unsere Variablen nicht die Koordinaten von Zürich an  
sondern die „Koordinaten“ des Tupels in unserem Raster

a= B2

denn Lat/long von Zürich sind in B2

b= a

# nun erstellen wir einen weiteren Pointer, er ist gleich (=) wie a

also: **b= B2**

|   | A   | B                 | C   |
|---|-----|-------------------|-----|
| 1 | ... | ...               | ... |
| 2 | ... | [47.3769, 8.5417] | ... |
| 3 | ... | ...               | ... |



# Aliasing

- Das Ändern eines Wertes führt zu einer Änderung des Zeigers.

```
second_variable = 42 # changing the value
print("Value of first:", first_variable)
print("Reference of first:", id(first_variable))
print("Value of second:", second_variable)
print("Reference of second:", id(second_variable))
```

Value of first: PYTHON

Reference of first: 4349862704

Value of second: 42

Reference of second: 4308446800

- Änderungen an Elementen innerhalb einer Liste führen **nicht** zu einer Änderung des Zeigers.

```
l1 = ["a", "b", "c"]
l2 = l1
l1[1] = "d"
print(l2)
l2[1] = "e"
print(l1)
```

['a', 'd', 'c']

['a', 'e', 'c']



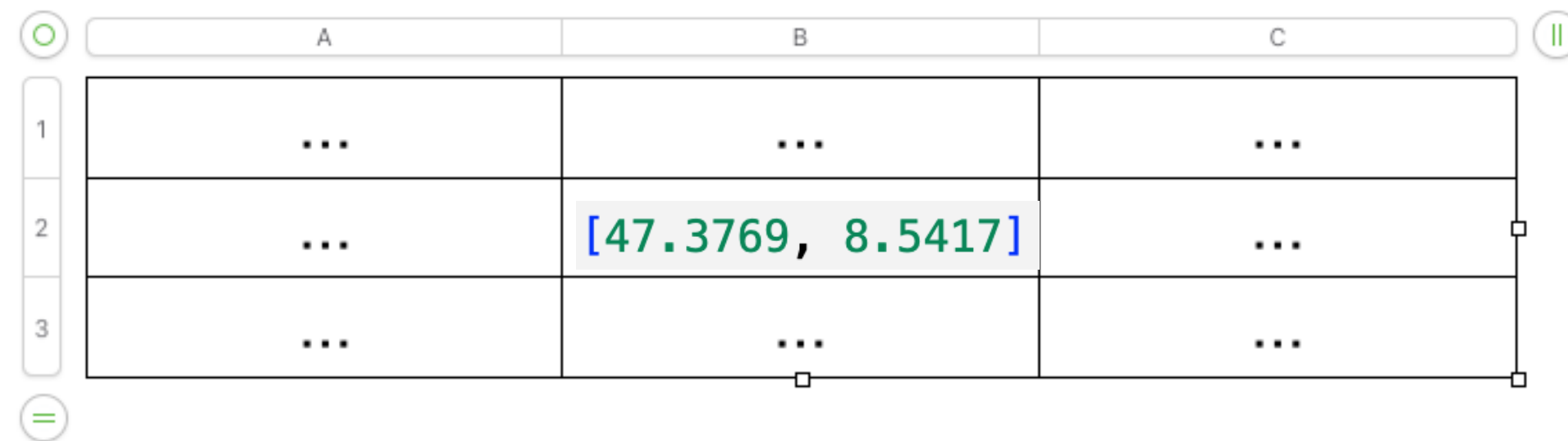
Änderungen an Elementen innerhalb einer Liste führen **nicht** zu einer Änderung des Zeigers.

a=B2

b=a #bedeutet, dass b=B2

wenn ich nun die **Lat/Long ändere**,  
bleibt meine **Liste aber immer noch in B2**

ich kann sozusagen über a oder über b meine **Liste** verändern



|   | A   | B                 | C   |
|---|-----|-------------------|-----|
| 1 | ... | ...               | ... |
| 2 | ... | [47.3769, 8.5417] | ... |
| 3 | ... | ...               | ... |

# Implikationen

**mutable Objekte innerhalb eines Immutable Containers können sich verändern!**

```
my_tuple = (1, 2, [3, 4])
print('previous id(my_tuple[2]):', id(my_tuple[2]))
my_tuple[2][0] = 10
print(my_tuple)
print('(still same) id(my_tuple[2]):', id(my_tuple[2]))
```

✓ 0.0s

```
previous id(my_tuple[2]): 4624529920
(1, 2, [10, 4])
(still same) id(my_tuple[2]): 4624529920
```

wir können die Liste selber verändern, denn das Tupel speichert immer noch denselben Pointer ab.

# Aliasing von Funktionen

- Aliasing gilt auch für Funktionen. Mittels Aliasing kann man bestehenden Funktionen neue Namen zuweisen.

```
def fun(name):  
    print(f"Hello {name}, welcome to Informatik II!!!")  
  
cheer = fun  
print("The id of fun():", id(fun))  
print("The id of cheer():", id(cheer))  
  
fun('everyone')  
cheer('students')
```

```
The id of fun(): 4408778960  
The id of cheer(): 4408778960  
Hello everyone, welcome to Informatik II!!!  
Hello students, welcome to Informatik II!!!
```



# Aliasing von Funktionen

- Aliasing kann auch auf Funktionen von Objekten angewendet werden.

```
class Test:
    def __init__(self):
        self._name = "original name"
    def name(self):
        return self._name
```

```
# create object of Test class
test = Test()
```

```
# create reference to name and its alias
name_fn = test.name          ## ()=> „original name“
name_fn_ref = name_fn        ## ()=> „original name“
```

```
print(name_fn())
```

```
test._name = "modified name"
```

hier wird das zugrundeliegende Objekt (nämlich test) verändert  
sowohl name\_fn\_ref() als auch name\_fn() würden nun „modiefied name“ zurückgeben!

```
print(name_fn_ref())
```

```
original name
```

```
modified name
```

name\_fn\_ref

# Aliasing: Quiz

- Was wird auf dem Bildschirm ausgegeben?

```
alist = [4, 2, 8, 6, 5]
blist = alist
blist[3] = 999
print(alist)
```

- A. [4, 2, 8, 6, 5]
- B. [4, 2, 8, 999, 5]

- Welche Vergleiche geben in Anbetracht der folgenden Listen 'True' aus? (Wähle alle, die zutreffen).

```
list1=[1,100,1000]
list2=[1,100,1000]
list3=list1
```

- A. `print(list1 == list2)`
- B. `print(list1 is list2)`
- C. `print(list1 is list3)`
- D. `print(list2 is not list3)`
- E. `print(list2 != list3)`

# Aliasing & Pointer

Pause? 



# Numpy

- Basics: eine Python Library, welche wir importieren müssen
- um schneller code zu schreiben, geben wir numpy den Spitznamen np

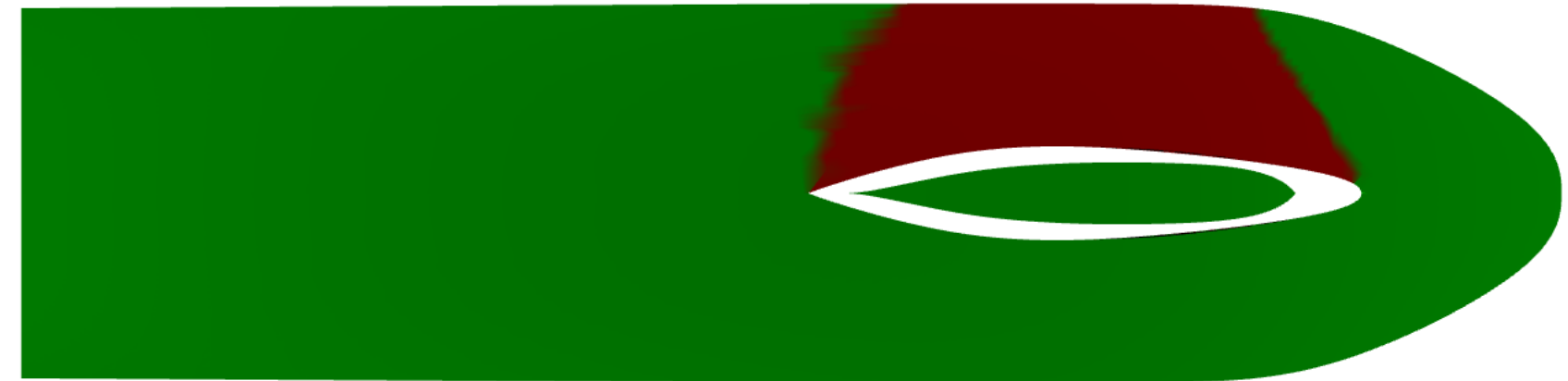
```
a = np.array([1, 2, 3, 4])  
b = np.array(range(5, 0, -1)) #[5, 4, 3, 2, 1]  
c = np.array([[1, 2], [3, 4]]) # two dimensional array
```

Erstellen eines Numpy Array mit `arange()`. Ähnlich wie bei `np.array(range())`.

```
a = np.arange(1, 5, 2) # [1, 3]  
b = np.arange(1, 5) # step = 1, [1, 2, 3, 4]  
c = np.arange(5) # start = 0, step = 1, [0, 1, 2, 3, 4]
```

# Numpy

grösstenteils wie Listen, nur schneller!

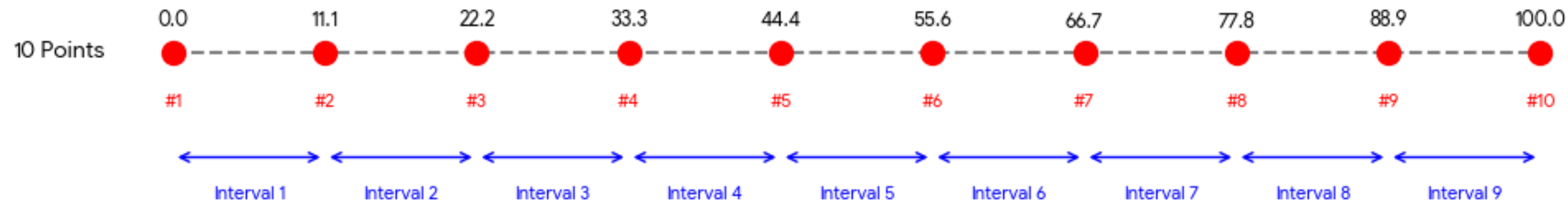


# np.arange() und np.linspace()

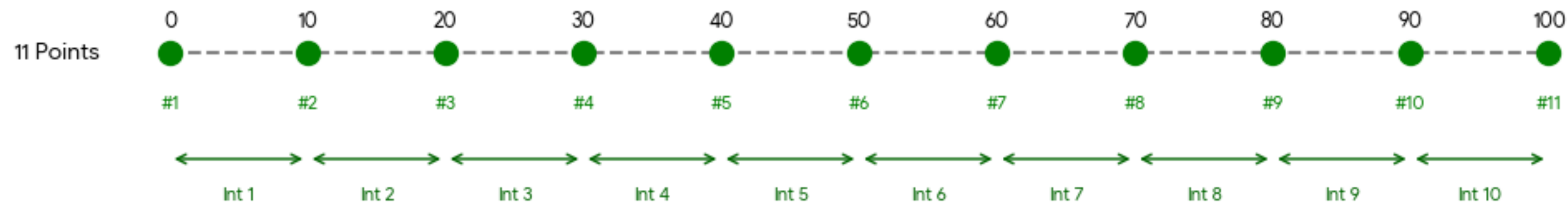
**arange() syntax:** np.arange(start, stop, step)

**linspace() syntax:** np.linspace(start, stop, num), # num default =50  
num ist die Anzahl „Punkte“ **inkl. start,stop!**

np.linspace(0, 100, 10) -> 9 Intervals



np.linspace(0, 100, 11) -> 10 Intervals (Clean!)





# Erstellung von Numpy Arrays & Quiz

Erstellen Sie ein Numpy-Array mit `linspace()`. Stopp ist **inklusive**.

```
a = np.linspace(2, 10, 5) # [2., 4., 6., 8., 10.]  
a = np.linspace(2, 100) # num = 50, [2., 4., 6., ..., 100.]
```

**Quiz:** Wie kann man das folgende Array mit Hilfe von `arange()` bzw. `linspace()` erzeugen?

```
array([5, 9, 13, 17])
```

# Numpy Array Operationen

Print

numpy arrays sind keine Listen!

```
print(np.array([2, 2, 6, 8])) #[2 2 6 8]
```

Länge und Größe

trotzdem: die Operationen sind meistens gleich

```
a = np.array([1, 2, 3, 4, 5, 6]) # one dimension
len(a) # 6
a.size # 6
b = np.array([[1, 2], [3, 4], [5, 6]]) # two dimensions
len(b) # 3
len(b[0]) # 2
b.size # 6
```

# Numpy Array Operationen

Zugriff auf ein Element

```
a = np.array([1, 2, 3, 4, 5, 6]) # one dimension  
a[2] # 3  
a[-2] # a[-2] = a[-2 + len(a)] = a[4] = 5
```

```
b = np.array([[1, 2], [3, 4], [5, 6]]) # two dimensions  
b[1] # array([3, 4])  
b[1, 0] # b[1, 0] = b[1][0] = 3
```



# „Slicing“ von Numpy arrays

**Syntax:** `new_array=array[start_row:end_row:row_step, start_col:end_col:col_step]`

- sozusagen ein slicing operator für die Zeilen
- und einen für die Spalten
- **vorgehen:**
  - mit Zeilen beginnen, identifizieren welche Zeilen enthalten sind
  - auf selektierte Zeilen den die Col Slicing Parameter anwenden
- **Syntax auf ZF empfohlen!**



# Gefilterte Listen-Abstraktion

Zerschneiden

|   | 0 | 1 | 2 |
|---|---|---|---|
| 0 | 1 | 2 | 3 |
| 1 | 4 | 5 | 6 |
| 2 | 7 | 8 | 9 |

```
A = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

# Gefilterte Listen-Abstraktion

Zerschneiden

|   | 0 | 1 | 2 |
|---|---|---|---|
| 0 | 1 | 2 | 3 |
| 1 | 4 | 5 | 6 |
| 2 | 7 | 8 | 9 |

```
A = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

```
A[1] # = A[1,:] = array([4, 5, 6]) (Zeile)
```

```
A[:, 2] # array([3, 6, 9]) (Spalte)
```

# Gefilterte Listen-Abstraktion

Zerschneiden

|   | 0 | 1 | 2 |
|---|---|---|---|
| 0 | 1 | 2 | 3 |
| 1 | 4 | 5 | 6 |
| 2 | 7 | 8 | 9 |

```
A = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

```
A[1] # = A[1,:] = array([4, 5, 6]) (Zeile)
```

```
A[:, 2] # array([3, 6, 9]) (Spalte)
```

```
A[0:2, 1:3] # array([[2, 3], [5, 6]])
```

„Zeilen 0 und 1, und die Spalten 1 und 3“ (Schnittmenge)



# Gefilterte Listen-Abstraktion

Zerschneiden

|   | 0 | 1 | 2 |
|---|---|---|---|
| 0 | 1 | 2 | 3 |
| 1 | 4 | 5 | 6 |
| 2 | 7 | 8 | 9 |

```
A = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

```
A[1] # = A[1,:] = array([4, 5, 6]) (Zeile)
```

```
A[:, 2] # array([3, 6, 9]) (Spalte)
```

```
A[0:2, 1:3] # array([[2, 3], [5, 6]])
```

```
A[1:2, ::2] # = A[1:2, 0:3:2] = [[4, 6]]
```

„zeile 1, und davon ein slicing mit start 0, stop len(), und step 2“

# Gefilterte Listen-Abstraktion

Zerschneiden

|   | 0 | 1 | 2 |
|---|---|---|---|
| 0 | 1 | 2 | 3 |
| 1 | 4 | 5 | 6 |
| 2 | 7 | 8 | 9 |

```
A = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

```
A[1] # = A[1,:] = array([4, 5, 6]) (Zeile)
```

```
A[:, 2] # array([3, 6, 9]) (Spalte)
```

```
A[0:2, 1:3] # array([[2, 3], [5, 6]])
```

```
A[1:2, ::2] # = A[1:2, 0:3:2] = [[4, 6]]
```

**Quiz:** `A[-3:3, ::2] = ?` #

# Numpy Array Statistiken

## Numpy Array Statistiken

```
a = np.array([5, 6, 7, 8, 1, 2, 3, 4])  
a.min() # 1  
a.max() # 8  
np.mean(a) # 4.5  
a.sum() # 36  
np.std(a) # standard deviation, 2.291
```

**super nützlich**

**Wichtig:** sowohl in CX als auch an der Prüfung Sicherheitscheck machen, ob erlaubt!



# Numpy Array Operationen

## Elementweise Operationen

```
A = np.array([[1, 2, 3], [4, 5, 6]])  
A + 1 # [[2, 3, 4], [5, 6, 7]]  
A * 3 # [[3, 6, 9], [12, 15, 18]]  
A ** 2 # [[1, 4, 9], [16, 25, 36]]  
np.sin(A) # [[sin(1), sin(2), sin(3)], [sin(4), sin(5), sin(6)]]  
B = np.array([[4, 5, 6], [7, 8, 9]])  
A + B # [[5, 7, 9], [11, 13, 15]]  
A * B # [[4, 10, 18], [28, 40, 54]]
```

## Matrix Multiplikation

```
A @ np.array([[1, 4], [3, 4], [4, 6]]) # [[19, 30], [43, 72]]
```



# Numpy Array Filterung & Quiz

```
a = np.arange(1, 10)
```

```
f = a % 3 == 0
```

```
a[f]
```

a: 

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|

f: 

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| F | F | T | F | F | T | F | F | T |
|---|---|---|---|---|---|---|---|---|

a[f]: 

|   |   |   |
|---|---|---|
| 3 | 6 | 9 |
|---|---|---|

**wichtiges Konzept!  
(Masking)**

## Quiz

```
a[a**2 < 20].sum() = ?
```

(originales a verwenden)

Solution:

**Numpy** 

**nurnoch einen kurzen Rekursions Refresher**

# Rekursion

- Eine Funktion wird als **rekursiv** bezeichnet, wenn sie sich selbst aufruft.
- Die Idee ist es ein großes Problem in **kleinere sich wiederholende Teile** desselben Problems aufzuteilen.
- Jeder rekursive Algorithmus beinhaltet mindestens 2 Fälle:
  - **Basisfall**: Ein einfaches Problem, das direkt beantwortet werden kann.
  - **Rekursiver Fall**: Ein komplexeres Auftreten des Problems, das nicht direkt beantwortet werden kann.
- Einige rekursive Algorithmen haben mehr als einen Basis- oder rekursiven Fall, aber alle haben mindestens einen von beiden.



# Rekursion: Beispiel Fakultät

- Die Fakultät einer Zahl ist das Produkt aller Zahlen von 1 bis zu dieser Zahl. Zum Beispiel ist die Fakultät von 6 (auch bezeichnet als 6!)  $1*2*3*4*5*6 = 720$ .
- Beispiel einer rekursiven Funktion zum Ermitteln der Fakultät einer Zahl:

```
def factorial(x):  
    if x == 1:  
        """base case"""  
        return 1  
    else:  
        """recursive case"""  
        return (x * factorial(x-1))  
  
num = 3  
print("The factorial of", num, "is", factorial(num))
```

The factorial of 3 is 6

```
factorial(3)      # 1st call with 3  
3 * factorial(2)  # 2nd call with 2  
3 * 2 * factorial(1) # 3rd call with 1  
3 * 2 * 1        # return from 3rd call as number=1  
3 * 2            # return from 2nd call  
6               # return from 1st call
```

**sollte bekannt sein oder??**

# Rekursion: Vor- und Nachteile

## Vorteile

- Rekursive Funktionen lassen den Code **sauber und elegant** aussehen.
- Komplexe Aufgaben können durch Rekursion in **einfachere Teilprobleme** zerlegt werden.

## Nachteile

- Rekursive Aufrufe sind meistens teuer (**ineffizient**), da sie viel Speicher und Zeit beanspruchen.
- Rekursive Funktionen sind **schwer zu debuggen**, da es manchmal schwierig ist, der Logik hinter der Rekursion zu folgen.

# Rekursion: Stack Overflow

- Jede rekursive Funktion muss eine **Grundbedingung** haben, die die Rekursion stoppt, sonst ruft sich die Funktion endlos selbst auf.
- Der Python-Interpreter **begrenzt** die Rekursionstiefe, um unendliche Rekursionen zu vermeiden die zu einem Stack Overflow führen.
- Standardmäßig beträgt die maximale Rekursionstiefe **1000**. Wird die Grenze überschritten, führt dies zu einem `RecursionError`.

```
def recursor():  
    recursor()  
recursor()
```

```
Cell In[1], line 2, in recursor()  
      1 def recursor():
```

```
----> 2     recursor()
```

```
RecursionError: maximum recursion depth exceeded
```



# Any() - Funktion

- returned einen bool der angibt ob ein wert mindestens einmal im container vorkommt.
- **sehr nützlich!!**

```
#example of the any function  
numbers = [1, 2, 3, 4, 5]  
print(any(num > 3 for num in numbers)) # True, because 4 and 5 are greater than 3
```

✓ 0.0s

True



# Exercise 2: Python II, fällig 9.März

## Bemerkungen& Tipps

- **String reverse:** „basis string-variable“ erstellen und die wie gefordert zusammenbauen, geht mit oder ohne Slicing. **Ganzzahlige Division**  $5//2 \Rightarrow 2$
- **Scalar Product:** easy, gute Anwendung für **Comprehension!**
- **Suchen:** Hinweis beachten, Slide dazu anschauen, schafft ihr in 5 Min! :)
- **Dict Comprehension:** Syntax anschauen, systematisch anwenden
- **List Comprehension:** Einiges einfacher als Dict-Comprehension!

# Lektionsübung: Einheitsmatrix

Eine **Einheitsmatrix** oder **Identitätsmatrix** ist eine quadratische Matrix, deren Elemente auf der Hauptdiagonale eins und überall sonst null sind.

Schreibe ein Python Programm welches:

- Die Größe  $n$  der Einheitsmatrix als Input nimmt und die Matrix als verschachtelte Liste ausgibt.
- Die Matrix muß nicht weiter formatiert sein. Jedoch muß man auf jedes Element der Matrix  $I$  mittels  $I[r][c]$  zugreifen können, wo  $r$  und  $c$  die Indices der Reihe und Spalte sind.

**Hinweise:** Listen können mit einer Konstanten multipliziert werden.

Außerdem, können List Comprehensions und Python's Ternäroperator zu einer kompakteren Lösung führen.

# Lektionsübung: Numpy Array

Code Expert — Numpy Array Slicing & Masking

Schreibe ein Python Programm für folgende Schritte:

- Erzeugen Sie ein zweidimensionales Feld (Aufgabe 1).
- Schneiden Sie das Array und berechnen Sie seine statistischen Ergebnisse (Aufgabe 2, 3).
- Filtern Sie das Array und berechnen Sie seine statistischen Ergebnisse (Aufgabe 4-6).

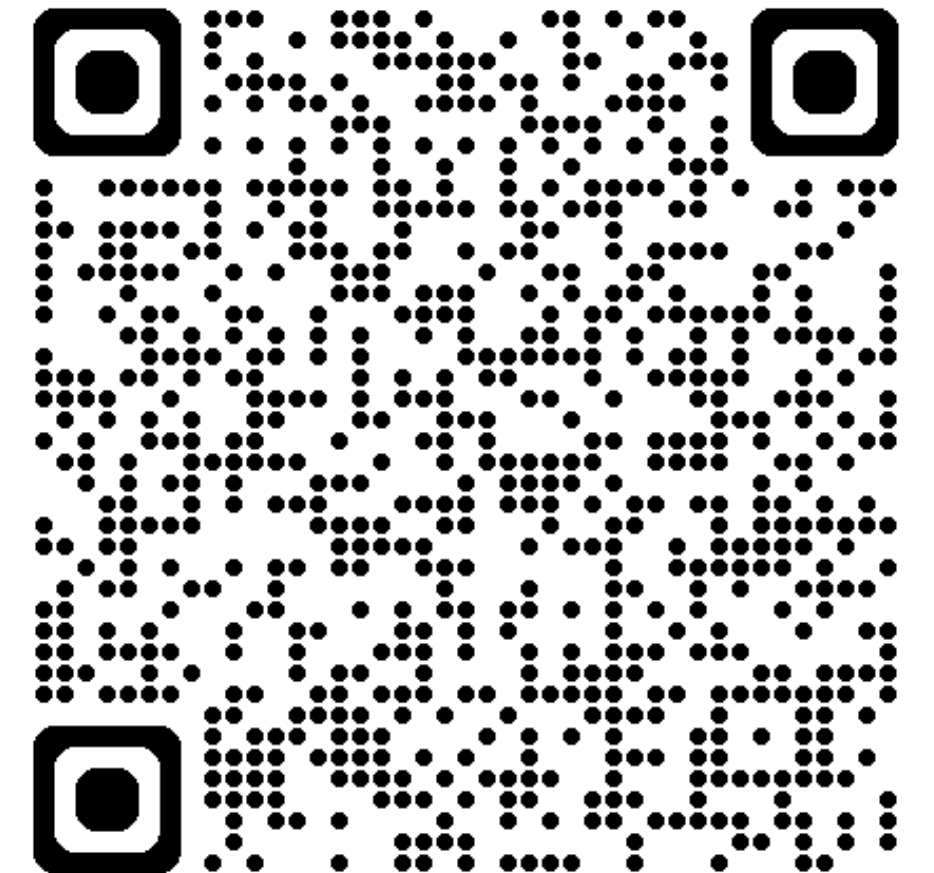
Siehe detaillierte Aufgabenbeschreibung in Code Expert.



# Bis nächste Woche :)

...nächste Woche:

- Pandas
- Matplotlib



Feedback sehr willkommen, einfach reinschreiben